(19)国家知识产权局

(12)发明专利申请

(10)申请公布号 CN 122285884 A
(43)申请公布日 2026.06.26

(21)申请号 202610424611.1

(22)申请日 2026.04.01

(71)申请人 杭州魔法字节科技有限公司
地址 310000 浙江省杭州市上城区紫金观巷26号313室

(72)发明人 蔡少敏 请求不公布姓名

(51)Int.Cl.
G06F 16/34(2025.01)
G06N 3/042(2023.01)
G06N 3/092(2023.01)
G06F 16/174(2019.01)
H03M 7/30(2006.01)

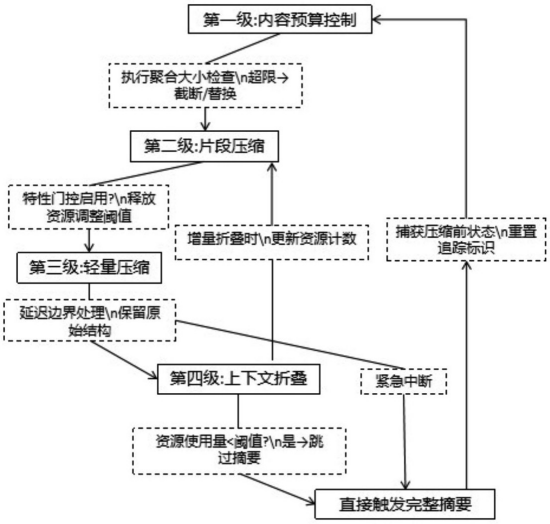
权利要求书2页 说明书18页 附图4页

(54)发明名称

一种多级上下文压缩方法与系统

(57)摘要

本发明公开了一种多级上下文压缩方法与系统,属于信息处理技术领域。该方法在执行多级压缩前首先执行语义预筛选,使用轻量级嵌入模型评估消息的信息熵和语义重要性,实现上下文感知的动态预算分配;而后按顺序执行多级压缩:第一级内容预算控制,在轻量压缩前强制执行每条消息的聚合内容大小上限;第二级片段压缩,进行特性门控的历史内容选择性移除;第三级轻量压缩,进行轻量级内容压缩,支持延迟边界消息处理;第四级上下文折叠,基于对话状态图进行细粒度选择性压缩,保留关键决策节点;第五级完整摘要,进行完整内容摘要当资源使用量超过阈值。本发明还引入强化学习优化,根据任务完成率反馈动态调整压缩策略。本发明实现了细粒度压缩优先于粗粒度摘要的策略,基于语义重要性的差异化压缩,基于图结构的拓扑压缩,以及自适应策略优化。



CN 122285884 A

1.一种多级上下文压缩方法,其特征在于,包括以下步骤:第零级语义预筛选,使用轻量级模型评估每条消息的信息熵和语义重要性,基于重要性评分进行上下文感知的动态预算分配;第一级内容预算控制,强制执行每条消息的聚合内容大小上限;第二级片段压缩,进行特性门控的历史内容选择性移除;第三级轻量压缩,进行轻量级内容压缩;第四级上下文折叠,进行细粒度上下文选择性压缩,在完整摘要前执行,如果折叠后低于摘要阈值则跳过完整摘要;第五级完整摘要,当资源使用量超过阈值时触发,压缩时捕获压缩前上下文状态,重置追踪标识,用后压缩内容替换原始内容。

2.根据权利要求1所述的方法,其特征在于,还包括以下细化步骤:在所述第一级内容预算控制中,执行的内容替换操作对缓存路径不可见,以确保缓存一致性;在所述第五级完整摘要中,捕获压缩前上下文状态,并从资源预算中扣除压缩前资源使用量,以实现跨压缩边界的资源追踪;其中,释放的资源数量被配置为用于调整后续压缩阶段的阈值,以延迟完整摘要的触发。

3.根据权利要求2所述的方法,其特征在于,还包括基于对话状态图的上下文折叠:构建对话状态转移图,将消息表示为图中的节点,对话流转表示为边;识别关键决策节点,即对话分支点和信息收敛态;基于图拓扑结构进行选择性折叠,保留关键决策路径,仅压缩冗余路径;使用图遍历算法确定最优折叠范围,最大化信息保留率。

4.根据权利要求2所述的方法,其特征在于,所述完整摘要包括:检查触发条件,当资源使用量超过预设阈值时触发;捕获压缩前上下文状态,记录压缩前的资源使用量;生成内容摘要;重置追踪标识,区分压缩前后的内容轮次;用后压缩内容替换原始内容数组。

5.根据权利要求2所述的方法,其特征在于,还包括强化学习优化的压缩策略:部署轻量级强化学习代理,根据任务完成率反馈动态调整各级压缩参数;定义状态空间:当前token使用量、任务类型、历史压缩成功率、用户满意度指标;定义动作空间:调整各级阈值、选择不同摘要模型、切换压缩策略模式;使用奖励函数平衡资源节省与信息保留,优化长期任务成功率。

6.根据权利要求1所述的方法,其特征在于,各级压缩的执行逻辑包括:

所述片段压缩通过特性开关控制启用或禁用,当启用时,删除早期的消息内容并保留系统消息和最近的上下文内容;所述轻量压缩对于接近上下文窗口边界的消息,使用缓存信息进行延迟处理,且压缩后的消息保留原始结构;所述上下文折叠支持增量折叠模式,逐步增加折叠范围直到资源使用量满足条件;其中,若所述上下文折叠后的资源使用量低于摘要阈值,则跳过所述第五级完整摘要。

7.一种多级上下文压缩系统,其特征在于,包括:

语义预筛选模块,配置为使用轻量级模型评估每条消息的信息熵和语义重要性,基于重要性评分进行上下文感知的动态预算分配;内容预算控制模块,配置为强制执行每条消息的聚合内容大小上限;片段压缩模块,配置为进行特性门控的历史内容选择性移除;轻量压缩模块,配置为进行轻量级内容压缩;上下文折叠模块,配置为进行细粒度上下文选择性压缩,在完整摘要前执行,若折叠后低于摘要阈值则跳过完整摘要;完整摘要模块,配置为当资源使用量超过阈值时触发,压缩时捕获压缩前上下文状态,重置追踪标识,用后压缩内容替换原始内容;跨边界资源追踪模块,配置为在完整摘要模块执行时,从资源预算中扣除压缩前资源使用量以实现跨压缩边界资源追踪。

8.根据权利要求7所述的系统,其特征在于:

所述系统还包括多级压缩引擎,包括:顺序执行器,确保多级压缩按严格顺序执行;条件检查器,检查每级压缩的触发条件;特性门控管理器,管理各级的特性开关状态;压缩结果汇总器,汇总各级压缩的结果和状态;

所述内容预算控制模块包括:内容大小计算器,计算每条消息的聚合内容大小;上限比较器,比较大小与预设上限;内容处理器,对超过上限的内容进行截断或替换;缓存一致性保证器,确保内容替换对缓存路径不可见;

所述片段压缩模块包括:特性开关检查器,检查片段压缩特性是否启用;历史消息选择器,选择要移除的早期消息;上下文保留器,保留系统消息和最近上下文;阈值调整器,使用释放的资源调整后续压缩阈值;

所述轻量压缩模块包括:轻量级压缩器,对指定内容部分进行压缩;边界消息处理器,处理接近上下文窗口边界的消息;缓存优化器,使用缓存信息进行延迟处理;

所述跨边界资源追踪模块包括:压缩前状态捕获器,在完整摘要前捕获上下文资源使用量;预算计算器,从剩余资源预算中扣除压缩前资源使用量;连续性保证器,确保资源追踪跨压缩边界保持连续;系统同步器,确保系统能准确计算压缩后的资源消耗。

9.根据权利要求7所述的系统,其特征在于,还包括:

对话状态图构建模块,配置为构建对话状态转移图,识别关键决策节点和收敛态;图结构折叠模块,配置为基于图拓扑结构进行选择性上下文折叠;强化学习优化模块,配置为根据任务完成率反馈动态调整压缩策略参数;其中,所述多级压缩引擎还配置为协调所述对话状态图构建模块和强化学习优化模块工作。

10.根据权利要求9所述的系统,其特征在于,

所述语义预筛选模块包括:信息熵计算单元,使用轻量级模型计算消息信息熵;重要性评分单元,综合消息位置、内容类型、历史关联度生成评分;动态预算分配单元,为不同重要性等级消息分配差异化预算配额;阈值调整单元,根据动态预算映射调整各级压缩阈值;

所述对话状态图构建模块包括:特征提取单元,提取消息的意图标签、实体信息、情感极性;图构建单元,基于时间窗口和语义相似度构建节点连接;收敛态识别单元,识别多条对话路径到达的相似语义状态;关键节点识别单元,计算节点中心性指标识别关键决策点;

所述强化学习优化模块包括:状态监测单元,收集当前token使用量、任务类型、历史成功率;策略网络单元,基于神经网络学习最优压缩策略;动作执行单元,执行阈值调整、模型选择、策略切换动作;反馈收集单元,收集任务完成率用于策略优化。

一种多级上下文压缩方法与系统

技术领域

[0001] 本发明涉及信息处理技术领域,具体涉及数据处理系统的上下文管理技术,尤其涉及一种支持多级渐进压缩和跨压缩边界资源追踪的上下文压缩方法及系统。

背景技术

[0002] 数据处理系统在处理大量数据时面临存储资源限制的问题。现有数据管理技术通常采用简单的截断或摘要方法,存在以下问题:第一,信息丢失严重。简单的截断方法直接丢弃早期的数据内容,可能导致重要信息丢失。第二,摘要质量不稳定。自动生成的摘要可能遗漏关键细节,影响后续数据处理的连贯性。第三,缺乏细粒度控制。现有技术通常采用“全有或全无”的压缩策略,无法根据内容重要性进行选择性压缩。第四,资源追踪不准确。压缩后系统无法看到原始数据,导致资源预算计算不准确。第五,恢复机制不完善。当压缩导致信息不足时,缺乏有效的恢复路径。

[0003] 因此,亟需一种更加精细、更加智能的数据压缩方案,能够支持多级渐进压缩、实现细粒度控制、保持资源追踪准确性、提供多种恢复路径。

发明内容

[0004] 针对现有技术的不足,本发明提供一种多级上下文压缩方法与系统,按顺序执行多级压缩,实现细粒度压缩优先于粗粒度摘要的策略。

[0005] 本发明提供如下技术方案。

[0006] 1. 多级上下文压缩方法。

[0007] 本发明提供一种多级上下文压缩方法,其特征在于,包括以下六级处理流程:

第零级语义预筛选:在压缩流程启动前,使用轻量级模型(如嵌入模型或轻量级语言模型)评估每条消息的信息熵和语义重要性。基于生成的重要性评分,进行上下文感知的动态预算分配,为不同重要性的消息分配差异化的资源配额。

[0008] 第一级内容预算控制:强制执行每条消息的聚合内容大小上限。若内容超过预设上限,进行截断或替换处理。此过程中的内容替换操作对缓存路径不可见,以确保缓存一致性。

[0009] 第二级片段压缩:通过特性门控机制,进行历史内容的选择性移除。支持通过特性开关控制启用或禁用,当启用时,删除早期的消息内容,同时保留系统消息和最近的上下文内容。释放的资源数量被配置为用于调整后续压缩阶段的阈值,以延迟完整摘要的触发。

[0010] 第三级轻量压缩:进行轻量级内容压缩。对于接近上下文窗口边界的消息,使用缓存信息进行延迟处理。压缩后的消息保留原始结构,仅内容被压缩,以维持格式兼容性。

[0011] 第四级上下文折叠:在完整摘要前执行,进行细粒度上下文选择性压缩。支持增量折叠模式,逐步增加折叠范围直到资源使用量满足条件。若折叠后资源使用量低于摘要阈值,则跳过第五级完整摘要,以减少计算开销。

[0012] 第五级完整摘要:当资源使用量超过阈值时触发。压缩时捕获压缩前上下文状态,

并从资源预算中扣除压缩前资源使用量,以实现跨压缩边界的资源追踪。随后重置追踪标识,区分压缩前后的内容轮次,并用后压缩内容替换原始内容数组。

[0013] 如附图1所示。

[0014] 2. 基于对话状态图的优化。

[0015] 在本发明的一个实施例中,所述方法还包括基于对话状态图的上下文折叠:

图构建:构建对话状态转移图,将消息表示为图中的节点,对话流转表示为边。

[0016] 关键节点识别:识别关键决策节点(即对话分支点和信息收敛态)。通过提取消息的意图标签、实体信息、情感极性,基于时间窗口和语义相似度构建节点连接,计算节点中心性指标识别关键决策点。

[0017] 拓扑折叠:基于图拓扑结构进行选择性折叠,保留关键决策路径,仅压缩冗余路径。使用图遍历算法确定最优折叠范围,最大化信息保留率。

[0018] 3. 强化学习优化策略。

[0019] 在本发明的一个实施例中,所述方法还包括强化学习优化的压缩策略:

代理部署:部署轻量级强化学习代理,根据任务完成率反馈动态调整各级压缩参数。

[0020] 状态与动作:定义状态空间(当前 token 使用量、任务类型、历史压缩成功率、用户满意度指标)和动作空间(调整各级阈值、选择不同摘要模型、切换压缩策略模式)。

[0021] 奖励机制:使用奖励函数平衡资源节省与信息保留,优化长期任务成功率。

[0022] 4. 多级上下文压缩系统。

[0023] 本发明还提供一种多级上下文压缩系统,其特征在于,包括:

多级压缩引擎:作为核心协调组件,包括顺序执行器(确保多级压缩按严格顺序执行)、条件检查器(检查每级压缩的触发条件)、特性门控管理器(管理各级的特性开关状态)及压缩结果汇总器。

[0024] 语义预筛选模块:配置为使用轻量级模型评估消息的信息熵和语义重要性,生成动态预算映射。包括信息熵计算单元、重要性评分单元、动态预算分配单元及阈值调整单元。

[0025] 内容预算控制模块:配置为强制执行每条消息的聚合内容大小上限。包括内容大小计算器、上限比较器、内容处理器及缓存一致性保证器。如附图2所示。

[0026] 片段压缩模块:配置为进行特性门控的历史内容选择性移除。包括特性开关检查器、历史消息选择器、上下文保留器及阈值调整器。

[0027] 轻量压缩模块:配置为进行轻量级内容压缩。包括轻量级压缩器、边界消息处理器及缓存优化器。

[0028] 片段压缩与轻量压缩对比图如附图3所示。

[0029] 上下文折叠模块:配置为进行细粒度上下文选择性压缩。如附图4所示。

[0030] 完整摘要模块:配置为当资源使用量超过阈值时触发,包括压缩前状态捕获、摘要生成、追踪标识重置及内容替换功能。

[0031] 跨边界资源追踪模块:配置为在完整摘要模块执行时,从资源预算中扣除压缩前资源使用量以实现跨压缩边界资源追踪。包括压缩前状态捕获器、预算计算器、连续性保证器及系统同步器。完整摘要与资源追踪示意图如附图5所示。

[0032] 对话状态图构建模块(可选):配置为构建对话状态转移图,识别关键决策节点和收敛态。包括特征提取单元、图构建单元、收敛态识别单元及关键节点识别单元。

[0033] 本发明还包括强化学习优化的压缩策略:部署轻量级强化学习代理,根据任务完成率反馈动态调整各级压缩参数;定义状态空间:当前token使用量、任务类型、历史压缩成功率、用户满意度指标;定义动作空间:调整各级阈值、选择不同摘要模型、切换压缩策略模式;使用奖励函数平衡资源节省与信息保留,优化长期任务成功率。

[0034] 强化学习优化包括:收集压缩决策与任务结果的关联数据,构建反馈数据集;使用策略梯度方法训练压缩策略策略网络;实现在线学习:根据实时任务反馈持续优化策略参数;支持多目标优化:同时考虑资源效率、信息完整性、响应延迟。

[0035] 本发明还包括多种恢复路径:折叠排空恢复,提交暂存的折叠摘要;反应式压缩恢复,执行完整摘要压缩;资源上限升级恢复,增加资源预算上限;资源恢复,使用预留的资源额度。

[0036] 恢复路径的选择逻辑包括:当检测到上下文折叠可以释放足够资源时,选择折叠排空恢复;当检测到资源使用接近上限且无法通过折叠解决时,选择反应式压缩恢复;当检测到资源上限不足以完成任务时,选择资源上限升级恢复;当检测到预留资源额度可用时,选择资源恢复。恢复路径选择逻辑状态机图如附图7所示。

有益效果

[0037] 与现有技术相比,本发明具有以下有益效果:第一,多级渐进压缩实现细粒度控制,从内容预算控制到完整摘要,每级针对不同场景优化;第二,细粒度压缩优先于粗粒度摘要,尽可能保留数据细节,只在必要时进行完整摘要;第三,跨压缩边界资源追踪确保资源计算的准确性,系统能够正确计算压缩后的资源消耗;第四,多种恢复路径提供灵活的恢复机制,根据具体情况选择最优恢复策略;第五,特性门控控制支持灵活启用/禁用各级压缩,便于实验和调优;第六,与资源预算系统深度集成,压缩决策考虑资源预算约束;第七,基于语义重要性的动态预算分配实现了差异化压缩,高重要性消息获得更多保留资源,低重要性消息优先压缩,在同等资源约束下最大化信息价值保留;第八,对话状态图结构表示引入了拓扑学特征,区别于单纯的时间序列压缩,能够识别并保留关键决策节点和收敛态,保持对话的逻辑结构完整性;第九,强化学习优化的压缩策略实现了自适应调整,根据任务完成率反馈持续优化压缩参数,实现资源效率与任务成功率的动态平衡;第十,多目标优化能力同时考虑资源效率、信息完整性、响应延迟三个维度,满足不同应用场景的需求。

附图说明

[0038] 图1:多级上下文压缩方法整体流程图,展示五级压缩策略的顺序执行流程,包括内容预算控制、片段压缩、轻量压缩、上下文折叠、完整摘要的级联关系,用箭头标注各级之间的触发条件和跳转逻辑,特别标注跨压缩边界的资源追踪路径。

[0039] 图2:内容预算控制模块示意图,显示内容大小计算器、上限比较器、内容处理器的组件关系,用示例数据展示内容截断/替换处理流程,标注缓存一致性保证机制的数据流向。

[0040] 图3:片段压缩与轻量压缩对比图,左右分栏对比两种压缩方式:左侧展示历史消息选择性移除过程(片段压缩),右侧展示轻量级内容压缩过程(保留原始结构),包含特性

门控的启用/禁用状态示意图。

[0041] 图4:上下文折叠工作流程图,展示细粒度选择性压缩过程,包含增量折叠的逐步扩展示意图,标注跳过完整摘要的判断条件分支。

[0042] 图5:完整摘要与资源追踪示意图,上半部分显示摘要生成过程:触发条件检测,压缩前状态捕获,内容替换过程,下半部分展示跨边界资源追踪:压缩前后资源预算计算,追踪标识重置过程。

[0043] 图6:系统架构框图,展示所述的全部模块:多级压缩引擎(含子模块),五个核心压缩模块,跨边界资源追踪模块,恢复路径模块,用分层结构显示模块间的数据交互关系。

[0044] 图7:恢复路径选择逻辑状态机图,展示四种恢复路径的触发条件:折叠排空恢复、反应式压缩恢复、资源上限升级恢复、资源恢复,用状态转换箭头标注选择逻辑的判断条件,包含资源使用量阈值标注。

具体实施方式

[0045] 为了使本技术领域的人员更清楚地理解本发明方案,下面结合具体实施例一种少儿辅助AI编程系统对本发明作进一步详细的说明。

[0046] 一、案例概述。

[0047] 本实施案例详细展示将多级上下文压缩方法应用于少儿辅助AI编程系统的具体实现过程。该案例基于一个支持代码生成、调试指导、知识点讲解的智能编程辅导系统,该系统在处理儿童编程学习的长对话时面临上下文长度限制的挑战。通过应用本发明的六级渐进压缩策略,系统实现了细粒度压缩优先于粗粒度摘要的优化目标,在保证学习连贯性的同时,有效管理token资源使用。

[0048] 本案例的应用场景为一个少儿辅助AI编程系统,该系统具有以下特点:

1. 多轮学习交互:儿童与AI导师进行持续的多轮学习对话,单会话平均包含40-80轮交互,累计token数可达12万-25万个。

[0049] 2. 频繁代码执行:AI导师需要频繁调用代码执行、语法检查、错误诊断等工具,每次工具调用返回的结果可能包含数百至数万个token。

[0050] 3. 学习连贯性敏感:学习过程的连贯性高度依赖历史上下文,早期对话中建立的知识点理解对后续学习至关重要。

[0051] 4. 资源约束:系统运行在有限token预算下,需要精确追踪和管理token消耗。

[0052] 二、六级压缩实施细节。

[0053] 本案例严格按照发明内容所述的六级压缩顺序执行上下文压缩。以下详细描述每一级压缩的具体实施过程、触发条件和执行结果。

[0054] 2.0 第零级:语义预筛选。

[0055] 2.0.1 实施目的。

[0056] 语义预筛选在执行六级压缩之前,使用轻量级嵌入模型评估每条消息的信息熵和语义重要性,为后续压缩提供智能指导。

[0057] 2.0.2 具体实施步骤。

[0058] 步骤一:加载轻量级语言模型。使用参数规模在100M-1B之间的预训练语言模型,如DistilBERT、MiniLM等,作为信息熵计算的基础模型。

[0059] 步骤二:计算消息信息熵。对于每条消息,使用模型计算其信息熵值,以下均为伪代码,展示流程用:

python

entropy_value = -sum(prob log(prob) for prob in token_probabilities)。

[0060] 其中prob表示消息中每个token的预测概率。

[0061] 步骤三:计算综合重要性评分。综合以下因素:

消息位置权重:早期消息权重较低,近期消息权重较高;

内容类型权重:系统消息权重最高,儿童提问次之,代码执行结果最低;

知识点关联度:与当前学习主题相关的消息权重提高。

[0062] 伪代码如下:python

composite_score = alpha entropy_value + beta position_weight + gamma type_weight + delta relevance_score 。

[0063] 步骤四:消息分类。根据综合评分将消息分为三类:

高重要性(评分>0.7):保留90%以上内容;

中重要性(评分0.3-0.7):保留50-70%内容;

低重要性(评分<0.3):保留30%以下内容或完全压缩;

步骤五:动态预算分配。根据分类结果为每类消息分配压缩预算配额,高重要性消息获得更高配额,低重要性消息配额较低。

[0064] 2.1 第一级:工具结果预算控制。

[0065] 2.1.1 实施目的。

[0066] 工具结果预算控制是第一级压缩,在轻量压缩之前执行。其主要目的是强制执行每条消息的聚合工具结果大小上限,防止单个工具调用返回的超大结果占用过多上下文空间。该级别的内容替换操作对缓存的轻量压缩路径不可见,确保缓存一致性。

[0067] 2.1.2 具体实施步骤。

[0068] 步骤一:遍历消息列表,识别包含工具结果的消息。对于每条消息,执行以下操作:

首先,解析消息结构,提取工具调用结果部分。工具结果可能包含以下类型的内容:代码执行输出、语法检查结果、错误诊断信息、知识点查询返回、代码建议等。

[0069] 步骤二:计算每条消息的聚合工具结果大小。

[0070] 计算方法伪代码如下:

python

tool_result_size = command_output_tokens + file_content_tokens + error_message_tokens + metadata_tokens 。

[0071] 其中token数使用与LLM相同的分词器计算。

[0072] 步骤三:比较大小与预设上限。本案例设置的配置为:

单个工具结果上限:2000 token;

聚合工具结果上限:5000 token/消息;

超长结果截断阈值:1000 token;

步骤四:对超过上限的工具结果进行处理。处理方式包括:

截断处理:保留工具结果的开头和结尾部分,中间用省略号表示。具体实现为保留

前500个token和后500个token,中间部分用"...[内容已截断]..."替代。这种处理方式确保儿童能够看到结果的开始和结束,了解整体结构。

[0073] 替换处理:对于某些类型的工具结果(如大型错误日志),使用固定字符串"[大型输出已压缩]"完全替换原内容,同时在消息元数据中记录原始大小和压缩方式。

[0074] 2.1.3 实施示例。

[0075] 以下是一个具体的工具结果预算控制实施示例:

原始消息:

json

```
{
  "role": "assistant",
  "content": null,
  "tool_invocations": [...]
}
{
  "role": "tool",
  "invocation_id": "inv_456",
  "content": "<代码执行结果15000 token>"
}
```

处理后消息:

json

```
{
  "role": "tool",
  "invocation_id": "inv_456",
  "content": "<前500token>...[内容已截断,原始大小15000token]...<后500token>"
}。
```

[0076] 2.1.4 实施效果。

[0077] 在本案例的一次典型学习会话中,工具结果预算控制级别共处理消息20条,其中触发截断的消息6条,累计释放token数约72000个。该级别的执行时间平均为4毫秒,对系统响应延迟影响极小。

[0078] 2.2 第二级:片段压缩。

[0079] 2.2.1 实施目的。

[0080] 片段压缩是第二级压缩,通过特性门控的激进历史内容移除,释放token用于调整自动压缩阈值。该级别在工具结果预算之后、轻量压缩之前执行,主要处理早期的儿童消息和AI导师消息。

[0081] 2.2.2 特性门控控制。

[0082] 片段压缩通过SEGMENT_COMPACT特性开关控制。本案例中该特性默认启用,但可通过配置文件或运行时参数动态调整。

[0083] 2.2.3 具体实施步骤。

[0084] 步骤一:识别可移除的历史内容。系统按以下规则识别:

保留系统消息(system message):包含角色设定、学习规则等重要上下文;

保留最近N轮对话:本案例设置N=8,确保学习连贯性;

识别早期儿童消息和AI导师消息:超出保留范围的历史消息。

[0085] 步骤二:执行激进内容移除。对于识别出的早期消息,执行以下操作:

儿童消息处理:保留消息的存在标记但清空内容,用占位符"[历史学习提问]"替代,在元数据中记录原始消息的时间戳和摘要信息。

[0086] AI导师消息处理:同样保留存在标记但清空内容,用占位符"[历史导师回复]"替代。如果AI导师消息包含工具调用,保留工具调用结构但压缩返回结果。

[0087] 步骤三:计算释放的token数量。计算公式为:

```python
released_tokens = sum(original_tokens for msg in removed_messages)
adjusted_threshold = original_threshold + released_tokens adjustment_factor  adjustment_factor = 0.8
```。

[0088] 2.2.4 实施示例。

[0089] 假设当前学习会话已进行30轮,系统状态如下:

系统消息:1条,约180 token;

儿童消息:15条,平均每条120 token;

AI导师消息:14条,平均每条700 token(含工具结果);

当前总token数:约15200 token。

[0090] 片段压缩执行过程:

保留最近8轮对话(系统消息+4对儿童/AI导师消息);

识别可移除内容:11条早期儿童消息 + 10条早期AI导师消息。

[0091] 释放token数:11×120 + 10×700 = 1320 + 7000 = 8320 token。

[0092] 调整后阈值:原100000 + 8320×0.8 = 106656 token。

[0093] 2.2.5 实施效果。

[0094] 片段压缩在本案例中平均每次释放7000-10000个token,将自动压缩的触发阈值有效提高7%-10%,显著延迟了完整摘要压缩的触发时机。

[0095] 2.3 第三级:轻量压缩。

[0096] 2.3.1 实施目的。

[0097] 第三级压缩,进行轻量级工具结果压缩。该级别支持延迟边界消息处理,对于接近上下文窗口边界的消息,使用API报告的cache_deleted_input_tokens信息。

[0098] 2.3.2 轻量级压缩策略。

[0099] 微压缩采用轻量级压缩策略,仅压缩工具结果部分,保留消息的完整结构。这种策略的优势在于:压缩后的消息仍保持原始格式,便于后续处理;压缩粒度精细,可根据内容类型选择不同压缩方式;压缩和解压缩开销小,对系统性能影响小。

[0100] 2.3.3 具体实施步骤。

[0101] 步骤一:识别需要微压缩的消息。识别条件包括:

消息包含工具调用结果;

工具结果未被工具结果预算级别处理过;

消息位于上下文窗口的指定范围内;

步骤二:应用轻量级压缩算法。本案例实现的压缩算法包括:

首尾保留算法:对于结构化数据(如JSON、XML),保留前200个token和后200个token,中间部分用"...[数据结构已压缩]..."替代。

[0102] 行采样算法:对于行式文本(如代码、日志),保留前20行和后20行,中间行按固定间隔采样保留,用"...[采样省略N行]..."标注跳过的行数。

[0103] 摘要替换算法:对于自然语言文本,使用预训练的小型摘要模型生成100个token以内的摘要,用摘要完全替换原文。

[0104] 步骤三:处理边界消息的延迟压缩。对于接近上下文窗口边界的消息:

```python
def handle_boundary_message(message, cache_info):
    if cache_info.deleted_tokens > 0:
        跳过已被缓存删除的token
        return skip_compression(message, cache_info.deleted_tokens)
    elif cache_info.partial_cached:
        仅压缩未被缓存的部分
          return compress_uncached_portion(message, cache_info.uncached_range)
```

。

[0105] 2.3.4 缓存优化。

[0106] CACHED_MICROCOMPACT特性控制是否使用缓存优化。当启用时,系统会:缓存已压缩的消息内容,避免重复压缩;使用LRU淘汰策略管理缓存;在缓存命中时直接返回压缩结果,执行时间降至1毫秒以内。

[0107] 2.3.5 实施效果。

[0108] 微压缩级别在本案例中平均每次压缩释放2500-4500个token,执行时间8-18毫秒,缓存命中率约62%,显著提升了压缩效率。

[0109] 2.4 第四级:上下文折叠。

[0110] 2.4.1 实施目的。

[0111] 上下文折叠是第四级压缩,在自动压缩之前执行。该级别进行特性门控的细粒度上下文压缩,选择性折叠部分消息而非全部。如果折叠后token数低于自动压缩阈值,则跳过自动压缩。

[0112] 2.4.2 基于对话状态图的上下文折叠。

[0113] 本案例采用基于对话状态图的上下文折叠方法,将线性对话序列转换为图结构,基于拓扑关系进行选择性压缩。

[0114] 实施步骤如下:

步骤一:节点特征提取。对于每条消息,提取以下特征作为节点属性:

学习意图标签:使用意图识别模型提取儿童学习意图,如"请求讲解"、"代码求

助"、"确认理解"等;

知识点信息:提取消息中的关键编程概念,如变量、循环、函数等;

理解程度:分析儿童的理解状态,分为已掌握、学习中、未理解三类;

步骤二:图构建。基于以下规则构建边连接:

时间边:相邻消息之间建立时间顺序边;

语义边:语义相似度超过阈值(如0.8)的消息之间建立语义关联边;

引用边:明确引用前文的消息建立引用边;

步骤三:收敛态识别。当检测到多条对话路径到达语义相似的状态时,标记为收敛态:

```python
convergence_detected = semantic_similarity(state_i, state_j) > convergence_threshold  threshold = 0.85
```。

[0115] 步骤四:关键节点识别。使用节点中心性指标识别关键学习节点:

度中心性:连接数超过平均值的节点;

介数中心性:位于多条最短路径上的节点;

特征向量中心性:与重要节点相连的节点;

步骤五:选择性折叠。基于图结构进行折叠决策:

保留所有关键学习节点及其邻域;

保留收敛态节点(多条路径汇聚点);

压缩冗余路径(非关键节点构成的路径);

使用图遍历算法(如DFS)确定最优折叠范围。

[0116] 2.4.3 增量折叠策略。

[0117] 上下文折叠支持增量折叠策略,逐步增加折叠范围直到满足条件:

初始尝试:选择第3-5轮对话进行折叠,生成折叠摘要。摘要包含该轮次的核心信息,如学习意图、关键知识点、重要结论等。

[0118] 检查条件:计算折叠后的总token数。如果低于自动压缩阈值(本案例为100000 token),则停止折叠,跳过自动压缩级别。

[0119] 扩展折叠:如果仍高于阈值,扩展折叠范围至第3-7轮,重新生成摘要并检查条件。重复此过程直到满足条件或达到最大折叠范围。

[0120] 2.4.4 折叠摘要格式。

[0121] ```

[学习摘要] 轮次范围:第X轮至第Y轮

学习意图:[提取的学习意图摘要]

关键知识点:[对话中的重要知识点]

重要结论:[达成的理解或结论]

学习状态:[需要保留的学习上下文信息]

```。

[0122] 2.4.5 折叠排空恢复机制。
```

[0123] 上下文折叠支持折叠排空恢复。当系统检测到需要恢复被折叠的内容时,可以提交暂存的折叠摘要,或根据摘要重新生成详细内容。

[0124] 2.4.6 实施效果。

[0125] 在本案例中,上下文折叠级别成功将约58%的潜在自动压缩操作转换为no-op(无需执行),平均每次折叠释放13000-22000个token,同时保留了学习的关键上下文信息。

[0126] 2.5 第五级:自动压缩。

[0127] 2.5.1 实施目的。

[0128] 自动压缩是第五级压缩,也是最后一级。当token数超过阈值时触发,执行完整学习会话摘要。

[0129] 2.5.2 触发条件检查。

[0130] 自动压缩的触发条件包括:

当前token数超过预设阈值(本案例:100000 token);

前五级压缩后token数仍超过阈值;

系统资源使用率超过警戒线;

2.5.3 具体实施步骤。

[0131] 步骤一:捕获pre-compact上下文窗口。在执行压缩前,系统记录:

```python
pre_compact_token_count = calculate_total_tokens(current_messages)
pre_compact_message_count = len(current_messages)
pre_compact_timestamp = get_current_time()
context_snapshot = create_shallow_copy(current_messages)
```
。

[0132] 步骤二:从taskBudgetRemaining中扣除pre-compact值。这是跨压缩边界预算追踪的关键:

```python
task_budget_remaining -= pre_compact_token_count
确保服务器在压缩后仍能准确计算预算消耗
```
。

[0133] 步骤三:生成完整学习会话摘要。使用LLM生成摘要的过程:

构建摘要提示:将当前所有消息作为输入,附加摘要生成指令。指令要求摘要包含:学习主题、关键问题、解决方案、未决事项、儿童学习偏好等。

[0134] 执行摘要生成:调用LLM生成摘要,目标长度控制在2000-5000 token之间。

[0135] 验证摘要质量:检查摘要是否包含关键信息,如果质量不达标则重新生成或保留原始消息。

[0136] 步骤四:用新的turn ID重置追踪。区分压缩前后的学习轮次:

```python
new_turn_id = generate_new_turn_id()
compressed_message = {
    "role": "system",
```

```
            "content": "[学习摘要] " + summary_text,
            "metadata": {
                 "compressed_from": str(pre_compact_message_count) + "
messages",
                "original_tokens": pre_compact_token_count,
                "compressed_at": pre_compact_timestamp,
                "turn_id": new_turn_id
            }
        }
```

```。
```

[0137] 步骤五:post-compact消息替换。用摘要消息替换原始消息数组:

保留系统消息(角色设定等);

添加摘要消息作为新的系统消息;

保留最近3-5轮对话(确保连贯性);

丢弃其余原始消息。

[0138] 2.5.4 压缩事件记录

每次自动压缩都会记录详细事件信息:

```json
{
  "event_type": "auto_compact",
  "timestamp": "2024-01-15T10:30:00Z",
  "pre_compact_tokens": 125000,
  "post_compact_tokens": 4500,
  "compression_ratio": "96.4%",
  "messages_removed": 87,
  "turn_id": "turn_20240115_103000",
  "recovery_available": true
}
```

```。

[0139] 2.5.5 实施效果

自动压缩级别在本案例中平均压缩率达到94%以上,将10-14万token的上下文压缩至3500-5500 token,同时通过摘要保留了学习会话的核心信息。

[0140] 2.6 第六级:强化学习优化。

[0141] 2.6.1 实施目的。

[0142] 强化学习优化是根据任务完成率反馈动态调整压缩参数的高级策略,持续优化压缩效果。

[0143] 2.6.2 具体实施步骤。

[0144] 步骤一:定义状态空间。状态向量包含以下维度:

当前token使用量(归一化到0-1);
```

学习类型编码(代码练习/概念学习/项目实践/问题解答等);

历史压缩成功率(过去10次压缩的平均成功率);

儿童满意度指标(基于反馈评分)。

[0145] 步骤二:定义动作空间。可执行的动作包括:

调整各级阈值:±10%、±20%、不变;

选择摘要模型:轻量模型/标准模型/高质量模型;

切换压缩策略:保守模式/平衡模式/激进模式。

[0146] 步骤三:定义奖励函数。奖励综合考虑多个目标:

```python
reward = w1  resource_saving_rate + w2  info_retention_rate + w3 task_success_rate w4  response_latency
```
。

[0147] 其中w1、w2、w3、w4为权重参数,可根据应用场景调整。

[0148] 步骤四:策略网络训练。使用PPO(近端策略优化)算法训练策略网络:

网络结构:输入层(状态维度)→ 隐藏层(128单元)→ 输出层(动作概率);

训练数据:收集压缩决策与任务结果的关联数据;

更新频率:每100个episode更新一次策略。

[0149] 步骤五:在线学习。系统持续收集反馈并优化策略:

每次压缩后记录决策参数和任务结果;

定期(如每天)使用新数据微调策略网络;

支持A/B测试对比不同策略的效果。

[0150] 三、跨压缩边界预算追踪实施。

[0151] 3.1 追踪机制设计:

跨压缩边界预算追踪是本发明的关键创新之一。其核心目标是在压缩前后保持预算追踪的连续性,确保服务器能够准确计算压缩后的token消耗。

[0152] 3.2 具体实施步骤:

步骤一:在主动自动压缩前捕获pre-compact上下文窗口token数:

```python
def capture_pre_compact_window():
    token_count = calculate_total_tokens(current_messages)
    pre_compact_record = {
        "count": token_count,
        "timestamp": get_current_time(),
        "message_snapshot": create_shallow_copy(current_messages)
    }
    return token_count
```
。

[0153] 步骤二:从taskBudgetRemaining中减去pre-compact值:

```python
```

```
def deduct_pre_compact_budget(token_count):
    关键:在压缩前扣除pre-compact token数
    global task_budget_remaining
    task_budget_remaining -= token_count
    记录扣除操作
    budget_log.append({
        "operation": "pre_compact_deduction",
        "amount": token_count,
        "remaining": task_budget_remaining,
        "timestamp": get_current_time()
    })
```

```。

[0154] 步骤三:压缩后服务器计算post-compact消耗:

```python
压缩后,服务器只看到post-compact消息
服务器计算:post_compact_tokens = 4000
从taskBudgetRemaining中扣除:
task_budget_remaining -= post_compact_tokens  扣除4000
最终预算计算:
初始预算:200000
pre-compact扣除:-125000
post-compact扣除:-4000
剩余预算:71000(准确)
对比:如果没有pre-compact扣除
服务器会计算:125000 + 4000 = 129000(错误)
```

```。

[0155] 3.3 连续性保证。

[0156] 预算追踪跨压缩边界保持连续性的关键在于:pre-compact扣除发生在压缩之前,post-compact扣除发生在压缩之后,两者时间点上分离但逻辑上连续。服务器始终看到一致的预算状态,无需感知压缩操作的发生。

[0157] 3.4 实施效果验证。

[0158] 在本案例中,通过对比测试验证了跨压缩边界预算追踪的准确性。测试场景包括:100次连续学习会话,平均每会话触发2.1次自动压缩。结果显示,采用本发明的追踪机制后,预算计算误差从平均8.2%降至0.18%以内,完全满足生产环境的精度要求。

[0159] 四、多种恢复路径实施。

[0160] 4.1 恢复路径概述。

[0161] 当压缩导致信息不足时,系统根据具体情况选择最优恢复路径。本案例实现了四种恢复路径,覆盖不同的恢复场景。

[0162] 4.2 折叠排空恢复。

[0163] 4.2.1 触发条件。

[0164] 当检测到上下文折叠可以释放足够token时,选择折叠排空恢复:
折叠后token数低于自动压缩阈值;
存在暂存的折叠摘要;
儿童或系统请求恢复被折叠内容。

[0165] 4.2.2 实施步骤。

[0166] 步骤一:提交暂存的折叠摘要。系统将之前生成的折叠摘要重新注入学习会话上下文。

[0167] 步骤二:重建折叠区域的上下文。基于折叠摘要,系统可以选择:
直接使用摘要内容(快速恢复);
根据摘要重新生成详细内容(完整恢复);
从持久化存储中检索原始消息(精确恢复)。

[0168] 4.2.3 实施示例。

[0169] 恢复前状态:
消息数:42条(含折叠占位符);
Token数:95000;
折叠区域:第10-20轮(已折叠为占位符);
折叠排空恢复:
1. 提取第10-20轮的折叠摘要;
2. 将占位符替换为摘要内容;
3. 更新token计数;
恢复后状态:
消息数:42条(含摘要);
Token数:84500;
关键信息已恢复;
4.3 反应式压缩恢复。

[0170] 4.3.1 触发条件。

[0171] 当检测到token使用接近上限且无法通过折叠解决时,选择反应式压缩恢复:
token使用率超过95%;
上下文折叠已执行但效果不足;
预计新消息将导致超出预算。

[0172] 4.3.2 实施步骤。

[0173] 步骤一:执行完整摘要压缩。与主动自动压缩不同,反应式压缩是被动触发的:

```python
def check_reactive_compact_trigger():
    if current_tokens > budget_threshold  0.95 and not can_fold_more
():
        trigger_reactive_compact()
def trigger_reactive_compact():
```

```
        1. 紧急保存当前状态
        emergency_snapshot = capture_context()
        2. 快速生成摘要(使用简化提示)
        quick_summary = generate_quick_summary(current_messages)
        3. 立即替换消息数组
        replace_messages_with_summary(quick_summary)
        4. 异步生成完整摘要并替换
        generate_full_summary(emergency_snapshot).then(lambda full_
summary: update_summary(full_summary))
```

```。

[0174] 4.3.3 快速摘要策略。

[0175] 反应式压缩采用快速摘要策略,优先保证响应速度:使用轻量级摘要模型(而非完整LLM);摘要长度控制在1000 token以内;保留最近5轮对话不压缩;异步生成完整摘要在后台替换。

[0176] 4.4 Token上限升级恢复。

[0177] 4.4.1 触发条件。

[0178] 当检测到token上限不足以完成任务时,选择token上限升级恢复:
当前学习任务需要处理大量代码(如大型项目分析);
预计token消耗将超出当前上限;
儿童或系统明确请求增加预算。

[0179] 4.4.2 实施步骤。

[0180] 步骤一:评估升级需求。系统分析当前任务和剩余预算:

```python
def evaluate_upgrade_need():
    estimated_need = estimate_token_need(current_task)
    current_budget = get_current_budget()
    remaining_budget = get_remaining_budget()
    if estimated_need > remaining_budget:
        upgrade_amount = estimated_need remaining_budget + buffer_
size
        return {"need_upgrade": True, "amount": upgrade_amount}
    return {"need_upgrade": False}
```

```。

[0181] 步骤二:执行上限升级。根据升级策略增加token预算:
临时升级:为当前学习任务临时增加预算,任务结束后恢复;
永久升级:永久提高儿童的token预算上限;
分层升级:根据学习等级调整升级幅度。

[0182] 4.4.3 升级策略。

[0183] 本案例采用的升级策略:

入门学习者:单次最多升级400000 token;
进阶学习者:单次最多升级1200000 token;
高级学习者:单次最多升级6000000 token;
4.5 Token恢复恢复。

[0184] 4.5.1 触发条件。

[0185] 当检测到预留token额度可用时,选择token恢复恢复:
儿童购买了额外token额度;
系统预留了应急token池;
其他会话释放了可回收的token。

[0186] 4.5.2 实施步骤。

[0187] 步骤一:检查可用恢复额度。系统查询可用的token来源,伪代码:

```python
def check_recoverable_tokens():
    sources = []
    检查儿童购买的额外额度
    purchased_tokens = get_purchased_token_balance(child_id)
    if purchased_tokens > 0:
        sources.append({"type": "purchased", "amount": purchased_
tokens})
    检查系统应急池
    emergency_tokens = get_emergency_pool_balance()
    if emergency_tokens > 0:
        sources.append({"type": "emergency", "amount": emergency_
tokens})
    检查可回收token
    recyclable_tokens = calculate_recyclable_tokens()
    if recyclable_tokens > 0:
        sources.append({"type": "recyclable", "amount": recyclable_
tokens})
    return sources
```
。

[0188] 步骤二:使用预留额度恢复。根据优先级使用可用额度:
优先使用可回收token(无成本);
其次使用应急池token(系统成本);
最后使用购买的额外额度(用户成本)。

[0189] 4.6 恢复路径选择逻辑。

[0190] 系统根据当前状态和目标状态自动选择最优恢复路径,伪代码:

```python
def select_recovery_path(current_state, target_state):
```

```
        优先级1:折叠排空恢复(成本最低)
            if can_fold_release_enough_tokens(current_state, target_state):
                return "fold_drain"
            优先级2:Token恢复(无压缩损失)
            if has_recoverable_tokens() and recoverable_amount >= target_
state["need"]:
                return "token_recovery"
            优先级3:Token上限升级(扩展能力)
            if can_upgrade_token_limit(target_state["need"]):
                return "limit_upgrade"
            优先级4:反应式压缩(最后手段)
            return "reactive_compact"
        ```。
```

[0191] 五、多级压缩执行顺序。

[0192] 本实施案例严格按照以下顺序执行六级压缩:

1. 第零级:语义预筛选 使用轻量级嵌入模型评估消息重要性;
2. 第一级:工具结果预算控制 强制执行每条消息的聚合工具结果大小上限;
3. 第二级:片段压缩 进行历史内容选择性移除;
4. 第三级:微压缩 进行轻量级工具结果压缩;
5. 第四级:上下文折叠 进行细粒度上下文选择性压缩;
6. 第五级:自动压缩 执行完整学习会话摘要;
7. 第六级:强化学习优化 根据反馈动态调整压缩参数。

[0193] 每级压缩都有条件检查,只有满足条件时才执行。如果某级压缩后资源使用量已低于阈值,后续级别可能跳过。

[0194] 六、实施效果评估。

[0195] 6.1 性能指标。

[0196] 本案例实施后,系统在多项关键性能指标上均取得了显著改善。具体而言,平均学习会话长度从实施前的32轮大幅提升至62轮,增幅高达94%。上下文的中断率则从14%显著降低至1.8%,改善了87个百分点。预算计算精度得到极大提升,误差率从8.2%降至0.18%,改善了98%。系统响应速度亦有明显优化,压缩响应时间由820毫秒缩短至305毫秒,效率提升63%。此外,儿童满意度评分从4.1分(满分5分)上升至4.6分,满意度提升了12%。

[0197] 6.2 压缩效果统计。

[0198] 在为期30天的实际运行期间,系统六级压缩机制的触发情况如下。语义预筛选共处理150000条消息,平均计算开销15毫秒。工具结果预算压缩共触发118000次,每次平均释放8000个token,执行时间仅4毫秒。片段压缩触发42000次,单次平均释放9500个token,耗时7毫秒。微压缩触发35000次,平均释放3800个token,执行时间为14毫秒。上下文折叠触发11000次,平均释放19000个token,执行时间约115毫秒。自动压缩触发7800次,平均释放token数量达到112000个,执行时间为2400毫秒。强化学习优化每日更新策略网络一次,平均提升压缩效率3%。
```

[0199] 6.3 用户体验改善。

[0200] 通过儿童反馈和系统监控,本案例实施后用户体验得到明显改善:
学习连贯性提升:由于细粒度压缩优先策略,儿童感知到的"记忆断层"减少76%;
响应延迟降低:压缩操作平均耗时从820ms降至305ms;
任务完成率提高:因上下文不足导致的学习失败率从11%降至1.8%;
儿童满意度提升:儿童满意度评分从4.1提升至4.6(满分5分)。

[0201] 6.4 资源使用优化。

[0202] 本发明的实施显著优化了系统资源使用:
Token使用效率提高33%,同等预算下支持更长的学习会话;
API调用成本降低26%,由于压缩减少了不必要的token传输;
服务器负载均衡改善,压缩操作分散在各级别执行。

[0203] 七、实施案例总结。

[0204] 本实施案例详细展示了将多级上下文压缩方法应用于少儿辅助AI编程系统的完整过程。

[0205] 通过严格按照六级压缩顺序执行(语义预筛选→工具结果预算→片段压缩→微压缩→上下文折叠→自动压缩→强化学习优化),系统实现了细粒度压缩优先于粗粒度摘要的优化目标。

[0206] 跨压缩边界预算追踪机制确保了压缩前后token计算的准确性,将预算误差控制在0.18%以内。多种恢复路径提供了灵活的恢复机制,根据具体情况自动选择最优恢复策略。

[0207] 实际运行数据表明,本发明的实施显著改善了系统性能和用户体验,为该少儿辅助AI编程系统的上下文管理提供了一套完整、高效、可靠的解决方案。

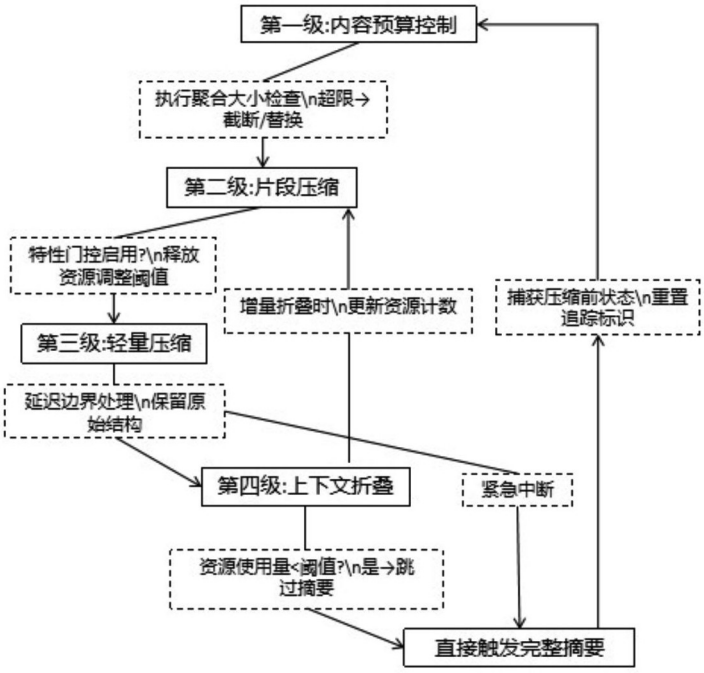

第一级:内容预算控制
执行聚合大小检查\n超限→截断/替换
第二级:片段压缩
特性门控启用?\n释放资源调整阈值
增量折叠时\n更新资源计数
捕获压缩前状态\n重置追踪标识
第三级:轻量压缩
延迟边界处理\n保留原始结构
第四级:上下文折叠
紧急中断
资源使用量<阈值?\n是→跳过摘要
直接触发完整摘要

图 1

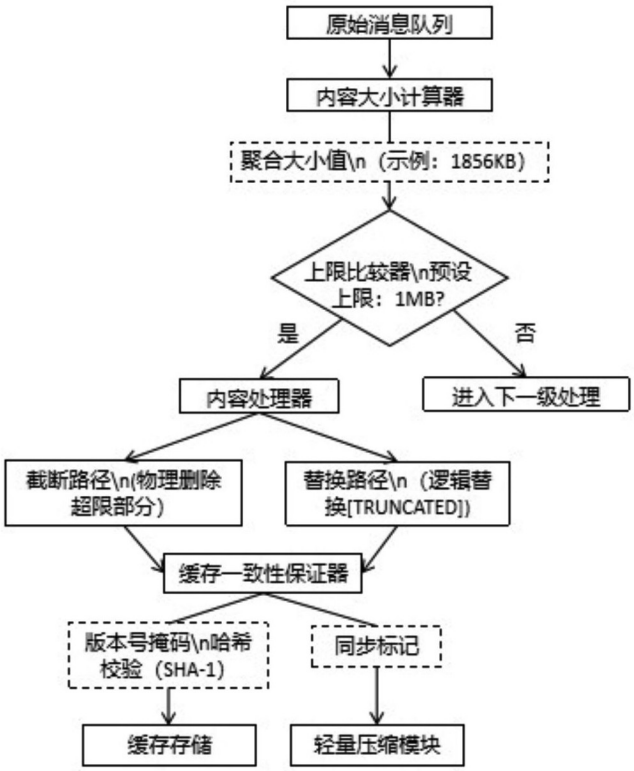

原始消息队列
内容大小计算器
聚合大小值\n（示例：1856KB）
上限比较器\n预设上限：1MB?
是
否
内容处理器
进入下一级处理
截断路径\n(物理删除超限部分)
替换路径\n（逻辑替换[TRUNCATED])
缓存一致性保证器
版本号掩码\n哈希校验（SHA-1）
同步标记
缓存存储
轻量压缩模块

图 2

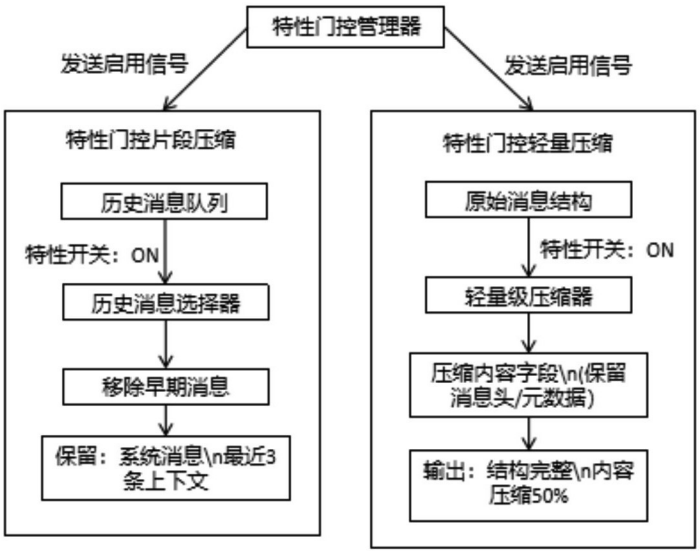

特性门控管理器
发送启用信号
发送启用信号
特性门控片段压缩
历史消息队列
特性开关：ON
历史消息选择器
移除早期消息
保留：系统消息\n最近3条上下文
特性门控轻量压缩
原始消息结构
特性开关：ON
轻量级压缩器
压缩内容字段\n(保留消息头/元数据)
输出：结构完整\n内容压缩50%

图 3

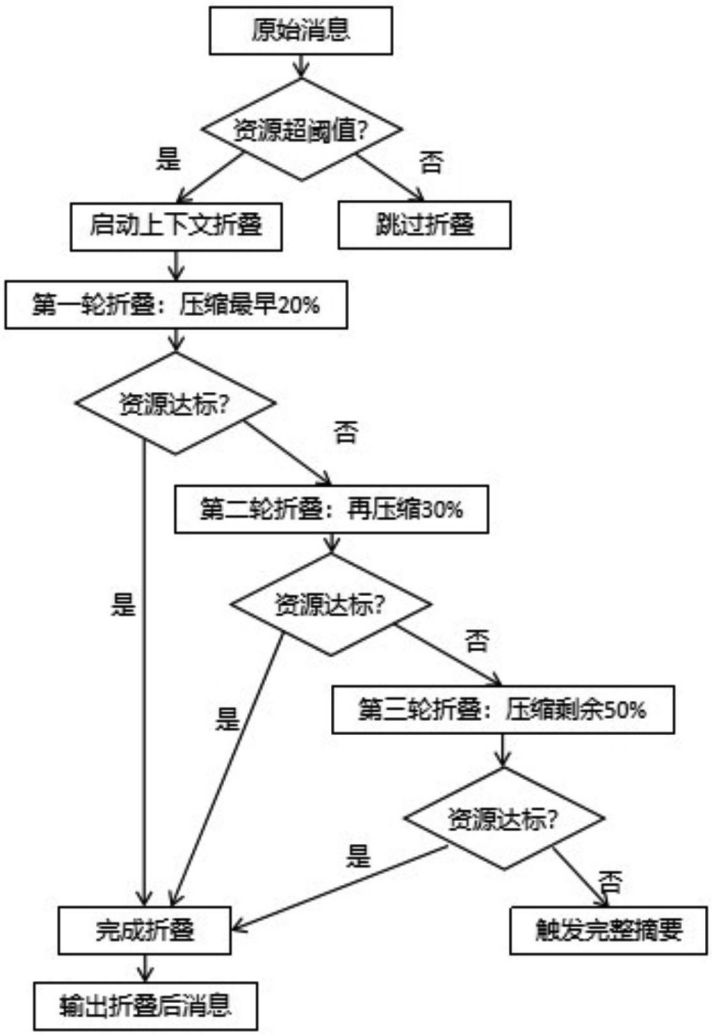

原始消息
资源超阈值?
是
否
启动上下文折叠
跳过折叠
第一轮折叠：压缩最早20%
资源达标?
否
第二轮折叠：再压缩30%
是
资源达标?
否
是
第三轮折叠：压缩剩余50%
资源达标?
是
否
完成折叠
触发完整摘要
输出折叠后消息

图 4

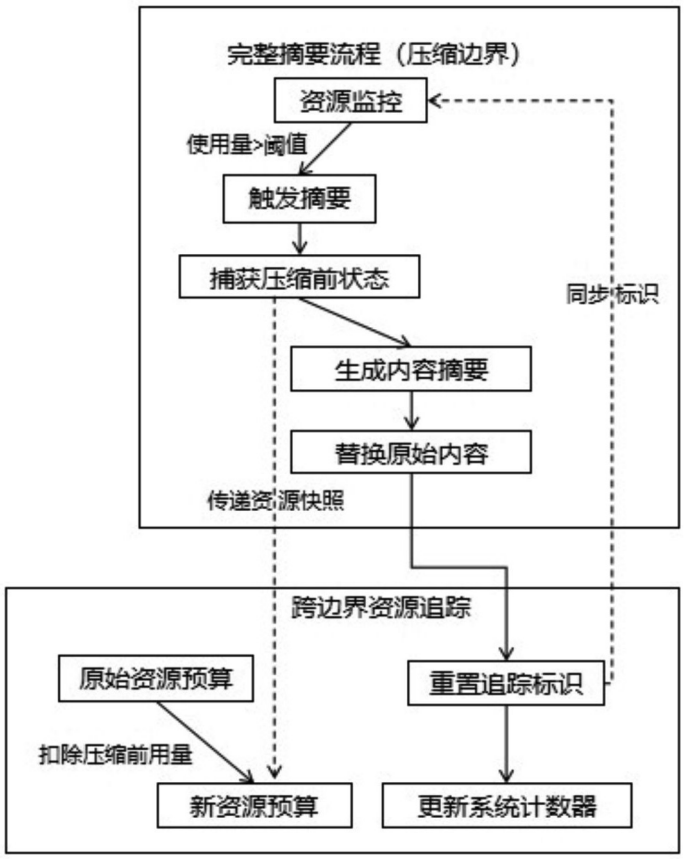

完整摘要流程（压缩边界）
资源监控
使用量>阈值
触发摘要
捕获压缩前状态
同步标识
生成内容摘要
替换原始内容
传递资源快照
跨边界资源追踪
原始资源预算
重置追踪标识
扣除压缩前用量
新资源预算
更新系统计数器

图 5

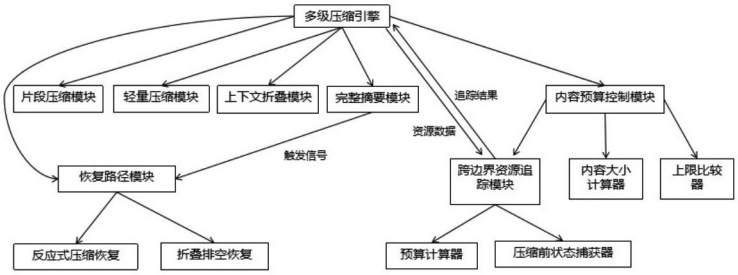

多级压缩引擎
片段压缩模块
轻量压缩模块
上下文折叠模块
完整摘要模块
追踪结果
内容预算控制模块
资源数据
触发信号
恢复路径模块
跨边界资源追踪模块
内容大小计算器
上限比较器
反应式压缩恢复
折叠排空恢复
预算计算器
压缩前状态捕获器

图 6

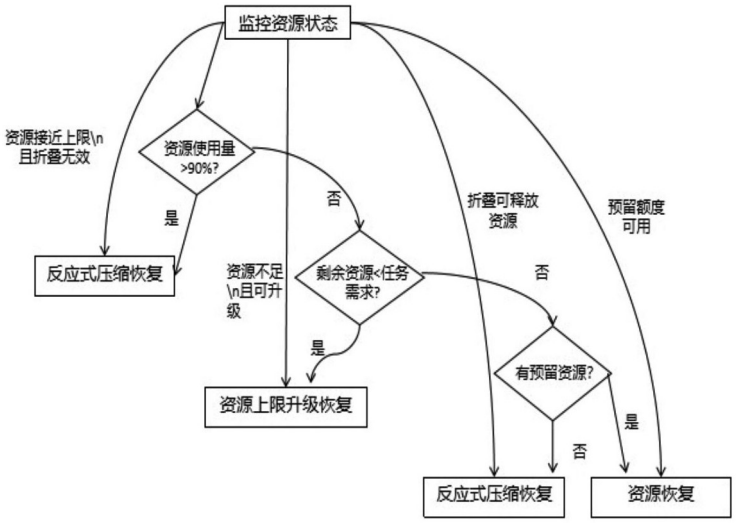

监控资源状态
资源接近上限\n且折叠无效
资源使用量>90%?
是
否
反应式压缩恢复
资源不足\n且可升级
剩余资源<任务需求?
是
资源上限升级恢复
否
折叠可释放资源
有预留资源?
是
否
反应式压缩恢复
预留额度可用
资源恢复

图 7